

Boggle Solver

The goal of this assignment was to create a solver for the game Boggle via Codio. The program needed to identify all valid words in a given dictionary formed on a NxN grid of letters. The base cases were that words must be formed using adjacent tiles (including diagonals), tiles cannot be reused within the same word, and special tiles such as “Qu”, “St”, and “Ie” count as two letters. I decided to approach this project by playing the Boggle game a few times seeing as I’ve never heard of it before. After understanding how the game was run I created methods to represent the sections essentially to the game (“setGrid“, ”setDictionary“, ”findWords“, ”getSolution“).

My approach to solving this problem was to use Depth-First Search (DFS) with backtracking. For each position on the board, I recursively looked for all possible paths. I did this by building strings and checking if they exist in the dictionary. I maintained a visited structure to ensure that no tile was reused in a single word path. Additionally, I designed the solution within a Boggle class as required, encapsulating the grid, dictionary, and results. To improve efficiency, I structured the dictionary in a way that allowed quick lookups and ensured that only words of length 3 or more were considered.

After pushing my assignment to SWE’s GitHub class, I added my reviewer to my file. When looking at my logic for having a title(letter) represent the length of the word and then remembering that length when moving to the next tile. Miles (reviewer) suggested I “ add a return statement below self.solutions.add(word) to stop unnecessary internal recursive calls!” Also within my setDictionary method when I’m making the code to ensure words under 3 letters are considered and that the grid has letters only. For readability he suggested I use the operator and instead of or not.

Within Miles' code I suggest that when initializing his dictionary as empty he should use a set not a list so I'd run faster. Lastly I recommended when checking if the word is in the grid, he could move the visited list out of the loops to save memory.

Based on the edits Miles gave me I implemented a return statement to make my program run faster. Adding this suggestion I stopped all the unnecessary recursive calls. That reduces the time complexity of the search making the solver significantly faster. I also changed my logic for the words less than or equal to three to be more simple. I used the and instead or and or which made the line itself short and less to comprehend.

When running my code on Codio in my terminal with Lint and Prettier, I found a lot of errors. Mainly they were due to Codio's natural indentation using Tab is two rather than the required four through Lint. I often had to manually indent (space button four times) to resolve. When making comments on my code I often forgot to add a space after the ('#'), I also had reached the limit on the line of code in words so I often had to move the comment to its own empty line. After unit testing I found that the `boggle_solver.py` I defined `getSolution` as `get_solution` later in code. I had to go through my code and change `get_solution` to `getSolution`. When unit testing I also checked if the solver can find the word "Queen" using only 4 tiles. The letters Qu and St should be considered as one tile not two separate letters. So within unit testing it's important the boggle solver follows this.

Through his assignment I learn about how to write code efficiently and focus more on the choices I make and not just getting my code to work. Through the code review, I got to think about how to simplify my code. Through Github workspace I learned how to move my code and push and pull it through Codio's terminal.

